Redes Neuronales

U4 Tópicos Avanzados

Héctor Maravillo

Section 1

Radial Basis Functions

Introduction

An enhancement to the standard multilayer perceptron techniques uses what are known as **radial basis functions (RBF)**.

These are a set of generally **non-linear functions** that are built up into one function that can partition the pattern space successfully.

The usual **multilayer perceptron** builds its classifications from **hyperplanes**, defined by the weighted sums

$$\sum_j w_{ij} x_i$$

which are arguments to nonlinear functions, whereas the **radial basis approach** uses **hyperellipsoids** to partition the pattern space.

Introduction

The **RBF** are defined by function of the form

$$\phi(||x - y||)$$

where $||\cdot||$ denotes some **distance measure**.

We can intuitively see that this expression describes some sort of **multi-dimensional ellipse**, since it represents a function, whose argument is related to a distance from a center, y.

The function s in k-dimensional space, which **partitions the space**, has elements s_k given by

$$s_k = \sum_{j=1}^{m} \lambda_{jk} \phi(||x - y_j||)$$

In other words, it is a **linear combination** of these **basis functions**.

Advantage

The **advantage** of using the radial basis approach is that once the **radial basis functions** have been chosen, all that is left to determine are the **coefficients** λ_j for each, to allow them to partition the space correctly.

Since these coefficients are added in a **linear** fashion, the problem is an **exact one** and has **a guaranteed solution** since there are no nasty local minima situations in which to fall.

In effect, the radial basis functions have expanded the inputs into a **higher-dimensional space** where they are now linearly separable.

Advantage

This approach is **guaranteed** to produce a function that **fits all** the data points, as long as there is a basis function **for each input** to be classified.

Having one basis function for each input does mean that **noisy** or **anomalous data points** will also be classified, however, and these will tend to cause distortion.

This **noise distortion** causes problems with **generalization**; since the classification surface is not necessarily smooth, very similar inputs may find themselves assigned to very different classes.

Advantage

The solution to this is to **reduce** the number of basis functions to a level at which an **acceptable fit** to the data is still achieved.

This means that the previously **exact problem** becomes one of **linear optimization**, but this is not a complex technique, and the classification surface will be smooth between the data points.

Gaussian function

The function ϕ is usually chosen to be a **Gaussian function**, i.e.

$$\phi(r) = e^{-r^2}$$

whilst the **distance measure** $||\cdot||$ is taken to be Euclidean:

$$||x - y|| = \sum_i (x_i - y_i)^2$$

where y represents the center of the hyperellipse

Network structure

The use of radial basis functions is becoming more popular, since they need only **linear optimization techniques**, which provide a guaranteed, globally optimal solution.

The difficulty in using them is in deciding on the set of basis functions to be used, in order to get an acceptable fit to the data.

Different from **Multilayer Perceptron networks**, which can be composed of several intermediate layers, the **RBF** typical structure is composed of only **one intermediate layer**, in which the activation function is **Gaussian**.

Network structure

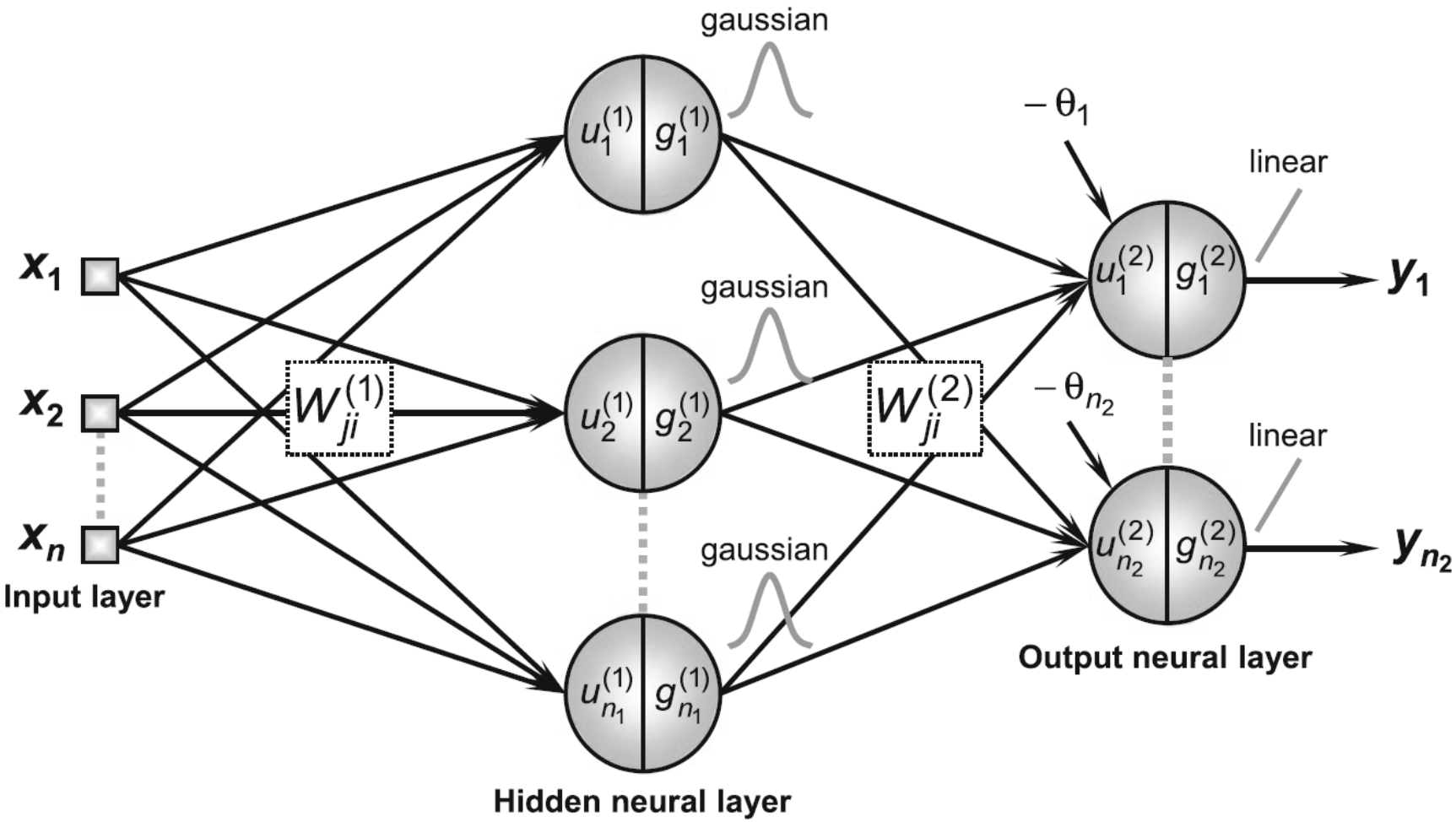

Figure: Typical configuration of an RBF network.

Training Process

Different from the **MLP**, the **training strategy** of the **RBF** is composed of two very distinct steps or stages:

- The **first stage**, associated with adjusting the weights of the neurons in the **intermediate layer**, adopts a **self-organized learning** method (**unsupervised**), which depends only on the features of the input data. This adjustment is directly related to the **allocation** of the radial basis functions.
- The **second stage**, related to the weight adjustment of the neurons in the **output layer**, uses a learning criterion similar to the criterion employed in the last layer of the MLP, that is, the **generalized delta rule**.

First stage

The neurons belonging to the **intermediate layer** of the **RBF** are composed of**activation functions with radial basis**, being the Gaussian one of the most used.

The expression that defines a **Gaussian activation function** is given by:

$$g(u) = e^{-\frac{(u-c)^2}{2\sigma^2}}$$

where c defines the **center** of the Gaussian function and σ^2 denotes its variance, which indicates how disperse is the activation potential u in relation to the center c.

So, the **free parameters** to be adjusted are the center c and the variance σ^2.

Gaussian function

The inputs $u_j^{(1)}$ of the first hidden layer are the **input vector** x, which represents the n external signals that are inputted in the network.

Consequently, the output of each neuron j of the intermediate layer is expressed by

$$g_j^{(1)}\left(u_j^{(1)}\right) = g_j^{(1)}\left(x^{(1)}\right) \quad (1)$$

$$= e^{-\frac{\sum_{i=1}^{n}\left(x_i - w_{ji}^{(1)}\right)^2}{2\sigma_j^2}} \quad (2)$$

where $j = 1, \ldots, n_1$.

Gaussian function

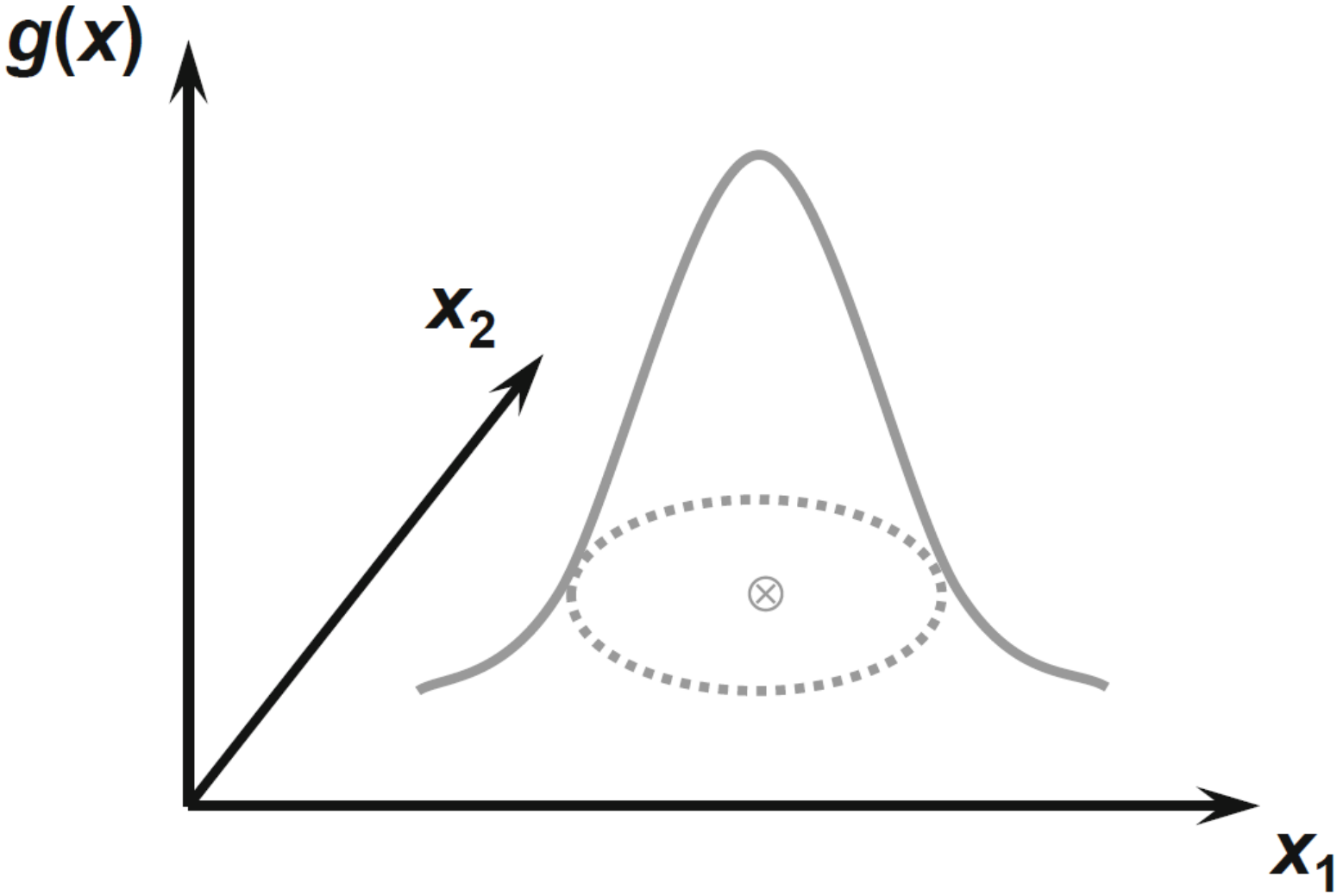

Figure: Gaussian radial basis function.

Geometric interpretation

Concerning problems of pattern classification, the Multilayer perceptron computes the delimiting boundaries between classes by combining **hyperplanes**.

In the case of the RBF with Gaussian activation function, the **delimiting boundaries** are defined by **hyperspherical domains**

Consequently, the pattern classification takes into account the radial distance of these patterns in relation to the center of these hyperspheres.

It is important to point out that these hyperspherical **domains** operate within the whole domain.

First stage

The primary objective of the training process related to the neurons from the **intermediate layer** is **to place the center** of their Gaussian functions in an adequate point.

One of the most used methods for this end is called the **k-means method**.

The value of the parameter k is equal to the **number of neurons** in the intermediate layer, because the activation function of each one of the neurons is a Gaussian.

Geometric interpretation

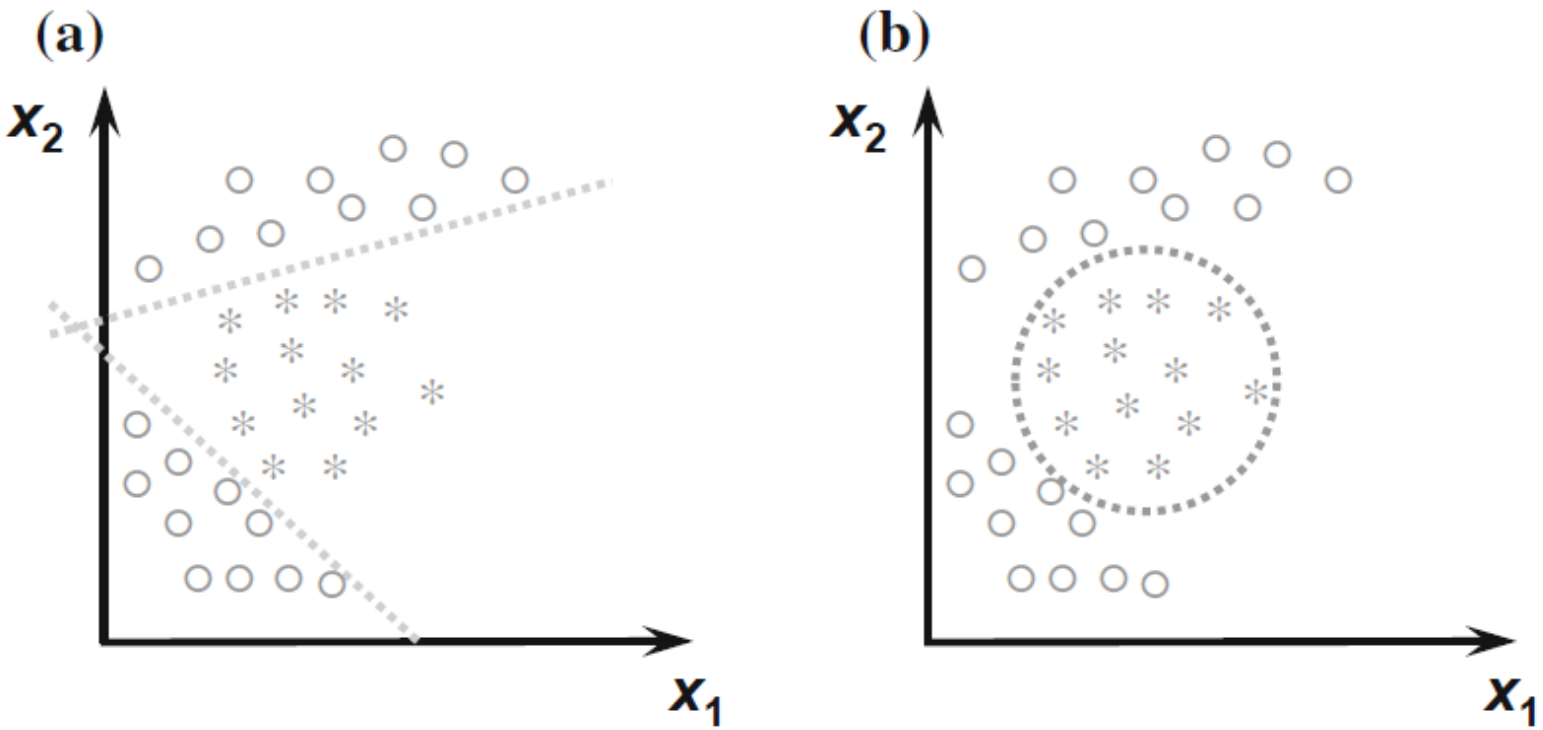

Figure: The Multilayer perceptron and RBF decision boundaries.

First stage

- (1) Obtain the set of training samples $\{x^{(k)}\}$
- (2) Initialize the weight vector of each neuron of the intermediate layer with the values of the n_1 first training samples.
- (3) Repeat
  - (3.1) For all samples, do
    - (3.1.1) Calculate the Euclidian distance between $x^{(k)}$ and $w_{ji}^{(1)}$, considering a single j-th neuron at each time.
    - (3.1.2) Select the neuron j with the shortest distance in order to group the given sample with the closest center
    - (3.1.3) Attribute $x^{(k)}$ to group $\Omega^{(j)}$
  - (3.2) For all $w_{ji}^{(1)}$ with $j = 1, \ldots, n_1$ do
    - (3.2.1) Adjust $w_{ji}^{(1)}$ according to the sample in $\Omega^{(j)}$:

$$w_{ji}^{(1)} = \frac{1}{m^{(j)}} \sum_{x^{(k)} \in \Omega^{(j)}} x^{(k)}$$

      with $m^{(j)}$ the number of samples in $\Omega^{(j)}$
  - Until: there are no more changes in the groups $\Omega^{(j)}$ between to iterations.

First stage

- (4) For all $w_{ji}^{(i)}$ with $j = 1, \ldots, n_1$ do:
  - Calculate the variance of each Gaussian activation function by using the mean squared distance criterion.

$$\sigma_j^2 = \frac{1}{m^{(j)}} \sum_{x^{(k)} \in \Omega^{(j)}} \sum_{i=1}^{n} \left(x_i^{(k)} - w_{ji}^{(1)} \right)^2$$

Second stage

The **second training stage** is performed by the same procedures used for training the **output layer** of the MLP.

Thus, different from the first training stage of the RBF, this second stage uses a **supervised learning** method.

Second stage

The training set to adjust the free parameters related to the output neurons are composed of pairs of input and desired outputs, in which the inputs are the **responses** produced by the **Gaussian functions** of the neurons in the intermediate layer when fed with training samples, that is:

$$u_j^{(2)} = \sum_{i=1}^{n_1} w_{ji}^{(2)} \cdot g_i^{(1)}(u_i^{(1)}) - \theta_j, \qquad j = 1, \ldots, n_2$$

where $w_{ji}^{(2)}$ and θ_j are respectively the weights and thresholds from the output layer neuron.

Second stage

Finally, similarly to the Multilayer perceptron, when a linear activation function is used in the output layer of the RBF, the output neurons only performs a **linear combination** of the **Gaussian functions** used by the neurons of the previous layer.

Conclusion

The application of the **first training stage** (**unsupervised**), followed by the **second stage** (**supervised**), allows the adjustment of all the **free parameters**

$$(w_{ji}^{(1)},\ \sigma_j^2,\ w_{ji}^{(2)},\ \theta_j)$$

of the RBF network.

Additionally, the **number of neurons** in the **intermediate layer** (**Gaussian activation functions**) can also be stipulated by using techniques of cross-validation.

Exercise

We consider the **XOR problem** and adopt an **RBF network** to perform the mapping to a linearly separable class problem.

Choose $k = 2$, the centers $c_1 = [1, 1]^T$, $c_2 = [0, 0]^T$, and

$$f(x) = \exp(-||x - c||^2)$$

The corresponding y resulting from the mapping is

$$y = y(x) = \begin{bmatrix} \exp(-||x - c_1||^2) \\ \exp(-||x - c_2||^2) \end{bmatrix}$$

Exercise

Hence

$$(0,0) \Rightarrow (0.135, 1) \quad (3)$$
$$(1,1) \Rightarrow (1, 0.135) \quad (4)$$
$$(1,0) \Rightarrow (0.368, 0.368) \quad (5)$$
$$(0,1) \Rightarrow (0.368, 0.368) \quad (6)$$

The resulting class position after the mapping in the y space, are now linearly separable and the straight line

$$g(y) = y_1 + y_2 - 1 = 0$$

as a possible solution.

Exercise

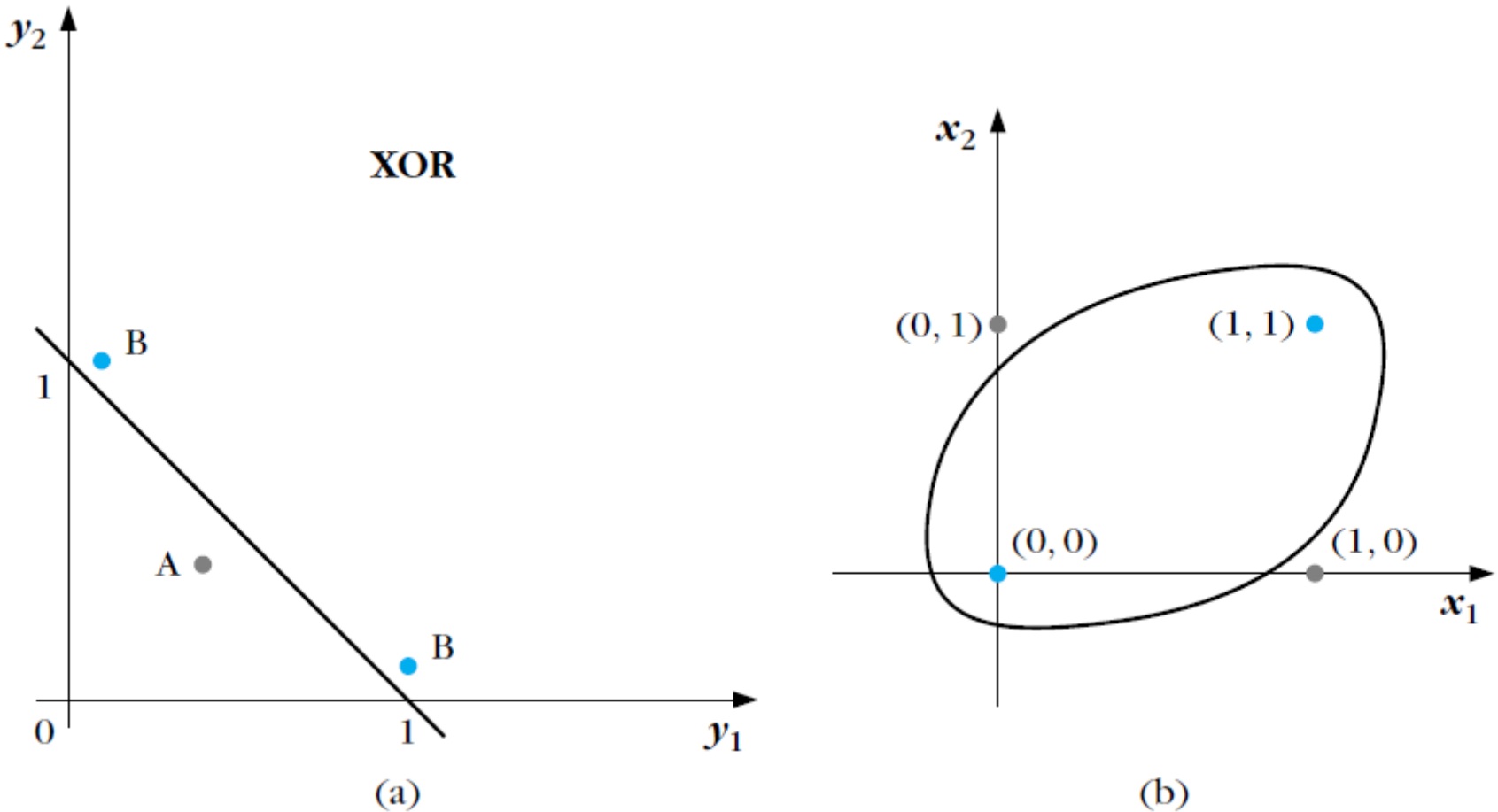

Figure: Decision curves formed by an RBF generalized linear classifier for the XOR task. The decision curve is linear in the transformed space (a) and nonlinear in the original space (b).

References

- Beale, R. & Jackson, T *Neural Computing: An Introduction*, Adam Hilger, 1990.
- Nunes da Silva, I., Hernane D., Andrade, R., Bartocci, L., and dos Reis, S. *Artificial Neural Networks. A Practical Course*, Springer, 2017.
- Theodoridis, S. & Koutroumbas, K. *Pattern Recognition*, 4th ed., Academic Press, 2009